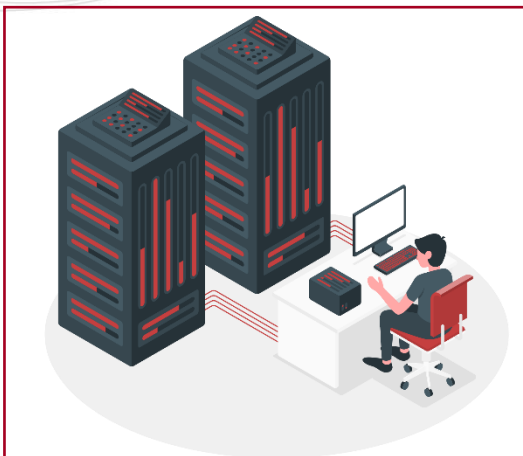




Universidad de El Salvador

Facultad Multidisciplinaria de Occidente
Ingeniería en Desarrollo de Software

Asignatura: **Administración de Bases de Datos**

Unidad #4

Fecha: 06 al 12 de octubre 2025

Tema: Operación y mantenibilidad.

Introducción:

Este material tiene como propósito introducirte en el manejo y la optimización de índices dentro de un sistema de gestión de bases de datos (DBMS), en particular PostgreSQL. A lo largo del contenido, aprenderás qué son los índices, cómo se utilizan para acelerar las consultas y por qué resultan esenciales en el rendimiento de las bases de datos. Se abordarán los diferentes tipos de índices disponibles, como B-tree, hash, GIN, GiST y compuestos, explicando en qué escenarios se recomienda cada uno. Además, se profundizará en la creación, mantenimiento, reorganización y reconstrucción de índices, así como en las ventajas y desventajas de su uso. Finalmente, conocerás técnicas para mantener los índices eficientes, incluyendo la gestión de índices parciales, únicos, de expresión y la diferencia entre reconstrucción con REINDEX y recreación mediante DROP y CREATE.

Sección 1: Manejo de índices

Índices de bases de datos

Antes de profundizar en los distintos tipos de índices y su aplicación práctica, es fundamental comprender qué es un índice en el contexto de las bases de datos y cuál es su importancia.

Un índice en una base de datos es una estructura de datos auxiliar que tiene como propósito principal acelerar las operaciones de búsqueda y recuperación de información, especialmente en tablas con una gran cantidad de registros. Los índices funcionan de manera similar a los índices de un libro, permiten localizar rápidamente la información deseada sin necesidad de leer página por página o en este caso registro por registro.

¿Cómo funcionan los índices?

Los índices mantienen una referencia organizada a los datos reales en las tablas. En esencia, son estructuras que apuntan a las posiciones físicas de las filas en las que se encuentra la información deseada. Esto se logra a través de algoritmos específicos de ordenamiento y búsqueda que permiten encontrar coincidencias más rápidamente que recorriendo la tabla completa.

Cuando no se utilizan índices el sistema de gestión de bases de datos (DBMS) realiza una búsqueda secuencial, examina uno a uno los registros de la tabla hasta encontrar aquellos que coincidan con el criterio de búsqueda. Esto en tablas grandes, se traduce en un alto consumo de recursos y mayor tiempo de ejecución.



Los índices permiten reemplazar este proceso por una búsqueda optimizada, reduciendo significativamente el número de comparaciones y accesos al disco necesarios para encontrar los datos.

Organización de los datos y tipos de tablas

Para entender mejor cómo los índices interactúan con las tablas, es importante conocer cómo están organizados físicamente los datos:

- **Tablas tipo heap:** Son estructuras sin un orden específico de almacenamiento, cada nuevo registro se inserta en el primer espacio disponible.
- **Tablas organizadas por índices:** El orden físico de los registros se basa en uno o varios campos indexados.

Un índice está vinculado a una o más columnas de una tabla, y cada entrada del índice contiene el valor de la(s) columna(s) y un puntero a la fila correspondiente. Este puntero es lo que permite que el sistema salte directamente al lugar donde se encuentra la información deseada.

Al insertar, actualizar o eliminar datos, el sistema también debe actualizar el índice, asegurando que su orden y consistencia se mantengan. Por esta razón, aunque los índices aceleran las búsquedas, también agregan un costo adicional en operaciones de escritura.

Ventajas y desventajas del uso de índices

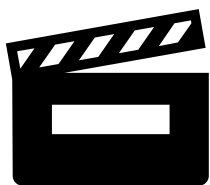
Ventajas:

- Mayor velocidad en las consultas: Especialmente en consultas con cláusulas WHERE, JOIN, ORDER BY, GROUP BY, etc.
- Mejor rendimiento en operaciones de lectura: Esto es crucial en aplicaciones donde predominan las consultas sobre la inserción o modificación de datos (sistemas OLAP).
- Facilitan la implementación de restricciones como UNIQUE o PRIMARY KEY.

Desventajas:

- Mayor consumo de espacio en disco: Cada índice adicional ocupa almacenamiento.
- Penalización en operaciones de escritura (INSERT, UPDATE, DELETE): Cada vez que se modifican los datos, el sistema también debe actualizar los índices asociados.
- Complejidad en el mantenimiento: Índices innecesarios o mal diseñados pueden disminuir el rendimiento en lugar de mejorarlo.

Es importante destacar que más índices no significa mejor rendimiento, crear índices innecesarios puede llevar a una sobrecarga en la base de datos y dificultar su mantenimiento.



Actividad Interactiva #1

Mira el video del link presentado a continuación y escribe en las líneas de abajo un resumen de 4 líneas de lo que consideres más importante.

<https://www.youtube.com/watch?v=Bp4TAaalljk>



Sección 2: Tipo de índices

Con PostgreSQL tenemos a nuestra disposición varios tipos de índices, siendo estos B-tree, Hash, GiST, SP-GiST, GIN, entre otros más especializados como BRIN y rum (en extensiones). Cada uno de los tipos de índice mencionados utiliza un algoritmo diferente que se adapta a distintos tipos de consultas y estructuras de datos. Comprender la naturaleza de cada índice nos permite mejorar significativamente el rendimiento de las consultas.

De forma predeterminada, el comando CREATE INDEX crea índices de tipo B-tree, que se adapta mejor a situaciones cotidianas. A continuación, veremos los tipos de índices más comunes en PostgreSQL y cómo usarlos de manera eficiente.

Índices de árbol B (B-tree)

Los índices de tipo B-tree son el estándar utilizado cuando creamos nuestros índices con la declaración CREATE INDEX. La "B" significa "Balanced" (Equilibrado), lo que implica que la estructura del árbol se mantiene balanceada para garantizar un rendimiento constante en las operaciones de búsqueda, inserción y eliminación.

Pueden operar con todo tipo de datos comparables, como enteros, cadenas, fechas, etc., y también pueden usarse para recuperar valores nulos. Este tipo de índice está diseñado para funcionar muy bien con el almacenamiento en caché y es ideal para operaciones como:

- Comparaciones de igualdad (=)
- Comparaciones de orden (<, >, <=, >=)
- Rango (BETWEEN, IN)

Ejemplo:

```
CREATE INDEX idx_alumno ON Alumnos (codAlumno);
```

Este índice permitirá búsquedas más rápidas en la columna codAlumno de la tabla Alumnos.

Índices Hash

En cuanto a los índices Hash, estos sólo son útiles para comparaciones de igualdad (=) y no admiten comparaciones de rango. Su desempeño puede ser superior a los B-tree cuando se trata de búsquedas exactas, pero tienen limitaciones:

- No se pueden utilizar para cláusulas ORDER BY.
- No se recomiendan para operaciones con grandes volúmenes de datos transaccionales.
- Hasta versiones recientes de PostgreSQL, no eran WAL-safe (lo que significa que no se replicaban ni recuperaban tras fallos), pero eso ha cambiado a partir de PostgreSQL 10.

Ejemplo:

```
CREATE INDEX idx_alumno ON Alumnos USING hash (codAlumno);
```

Índices GIN (Generalized Inverted Index)

Los índices GIN son ideales cuando una sola columna contiene múltiples valores, como los tipos de datos ARRAY, jsonb, y especialmente para búsqueda de texto completo. A diferencia de los índices B-tree, los GIN almacenan una lista invertida de elementos, lo que permite buscar eficientemente si un elemento está contenido en un conjunto.



Ejemplo:

```
CREATE INDEX busca_alumno_idx ON Alumnos  
USING gin (nome gin_trgm_ops, email gin_trgm_ops);
```

En este ejemplo, gin_trgm_ops se utiliza para habilitar búsquedas mediante trigramas, que permiten búsquedas parciales o aproximadas en cadenas de texto.

Trigrama: Una secuencia de tres letras consecutivas. Por ejemplo, "casa" genera los trigramas: cas, asa. Esto permite búsquedas flexibles y tolerantes a errores tipográficos.

Índices GiST (Generalized Search Tree)

El índice GiST permite construir estructuras de árbol equilibradas similares a B-tree, pero con mayor flexibilidad. Se emplean en casos más complejos como:

- Tipos de datos geométricos.
- Texto completo (como alternativa a GIN).
- Datos espaciales (como PostGIS).
- Búsquedas con rangos o similitudes.

Ejemplo:

```
CREATE INDEX busca_alumno_cep_idx ON Alumnos  
USING gist (cep gin_trgm_ops);
```

Aunque este ejemplo usa gin_trgm_ops con GiST, en la práctica, los operadores trgm son más comúnmente asociados con GIN. Este tipo de uso debe ser verificado según el contexto y tipo de dato.

Índices Concurrentes

Cuando se crean índices, la tabla se bloquea automáticamente para instrucciones de inserción, actualización y eliminación hasta que el índice se haya creado completamente. Sin embargo, crear índices en tablas grandes es una operación costosa que puede llevar mucho tiempo, lo que podría causar interrupciones en aplicaciones en producción.

Para solucionar este problema, PostgreSQL ofrece la opción de crear índices concurrentes usando la palabra clave CONCURRENTLY. Esta opción permite que otras operaciones de datos (como INSERT, UPDATE, DELETE) sigan ocurriendo mientras se construye el índice.

La sintaxis es la siguiente:

```
CREATE INDEX CONCURRENTLY index_name ON table_name USING btree (column);
```

Esto es especialmente útil en entornos de producción donde la disponibilidad es crítica.

Índices de una sola columna

El índice de una sola columna se utiliza básicamente cuando una tabla representa principalmente una sola categoría de datos, o incluso consultas que cubren solo una categoría en la tabla. Normalmente en el diseño de una base de datos, las tablas representan una única categoría de datos, por lo que normalmente se utiliza un índice de una sola columna y la sintaxis para este tipo de índice es la siguiente:

```
CREATE INDEX index_name ON table_name (column);
```



La eficiencia de este tipo de índice disminuye si se requiere evaluar varias columnas al mismo tiempo en una consulta, en cuyo caso es mejor usar un índice multicolumna (lo veremos más adelante).

Además, es importante destacar que PostgreSQL puede combinar varios índices de una sola columna durante una consulta, pero esto es menos eficiente que tener un índice multicolumna si la combinación es frecuente.

Ejemplo:

A modo de ejemplo sencillo, veamos a continuación cómo sería crear un índice para nuestra tabla de productos:

```
SELECT COUNT(*) FROM productos WHERE codigo_producto = 320;
```

Como podemos ver a continuación en la Ilustración 1, en esta consulta cuenta cuántos registros hay en la tabla productos que tienen el valor 320 en la columna codigo_producto. La cual al no tener un índice definido tardó 73 ms en realizar la consulta.

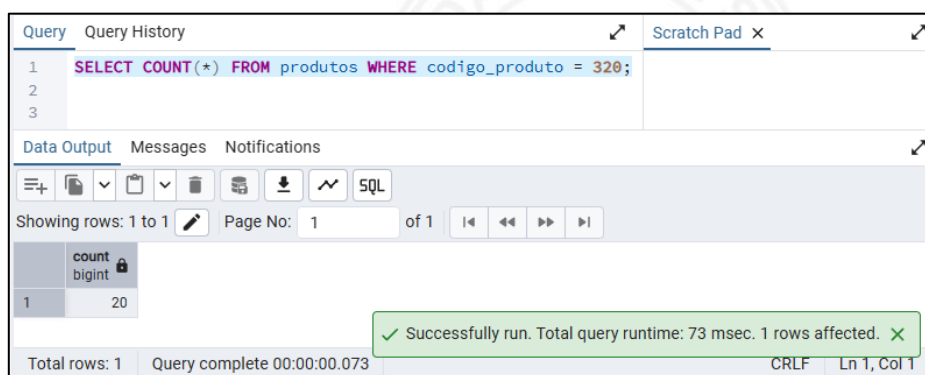


Ilustración 1 Consulta sin índice.

Si no tenemos un índice definido entonces tendremos un escaneo completo de la tabla, lo que será una operación costosa debido a la búsqueda de los registros en cuestión. Como podemos ver, solo se usó una columna junto con la cláusula WHERE, por lo que tenemos un índice en una sola columna, que es codigo_producto, en el caso de nuestra consulta.

Con el índice nuestra consulta se vuelve más optimizada, como podemos ver en la siguiente instrucción para crear el índice de nuestra consulta:

```
CREATE INDEX productos_index ON productos (codigo_producto);
```

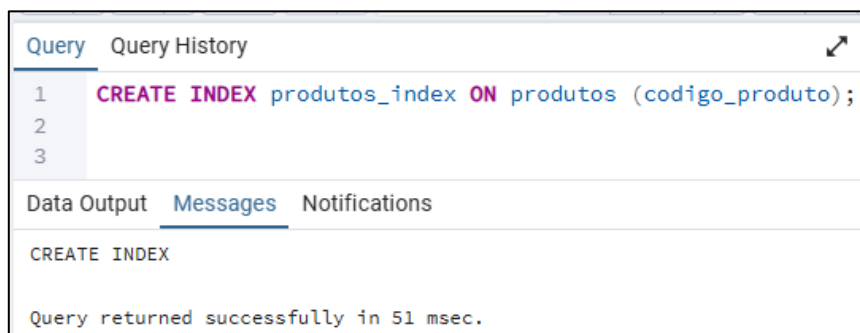
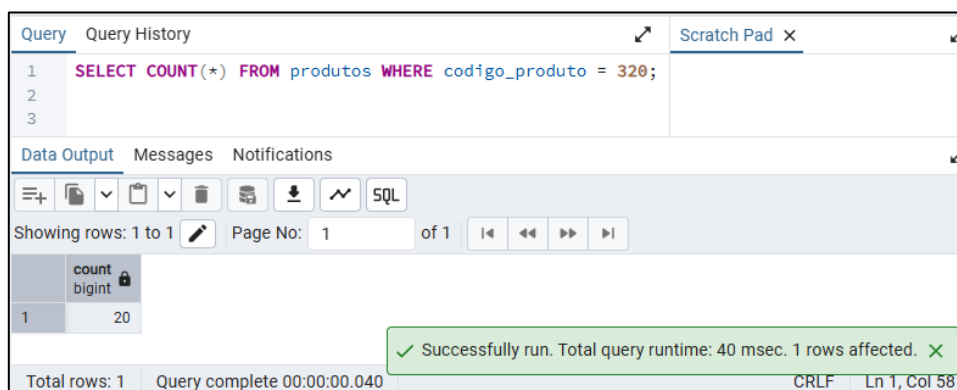


Ilustración 2 Crear el índice a una columna.



Hasta este punto, creamos un índice Btree, al que llamamos `produtos_index`, presente en la columna `codigo_producto` de la tabla "produtos". Realizada esta modificación intentaremos realizar nuevamente la consulta para que podamos ver el tiempo que se dedicará a esta operación, como podemos ver en la Ilustración 3.



Query		Query History	Scratch Pad
1	SELECT COUNT(*) FROM produtos WHERE codigo_producto = 320;		
2			
3			

Data Output		Messages	Notifications
Showing rows: 1 to 1 of 1 Page No: 1			
	count bigint		
1	20		

✓ Successfully run. Total query runtime: 40 msec. 1 rows affected. ✗

Total rows: 1 Query complete 00:00:00.040 CRLF Ln 1, Col 58

Ilustración 3 Resultado de la consulta con índice.

Como podemos ver, el resultado de la consulta con el índice tomó 33 ms menos, por lo que es mucho más rápido en comparación con el uso de consultas sin índices. La gran diferencia se debe a que consideramos una base de datos en la que tenemos miles de registros aproximadamente 10,000 y nos damos cuenta de que la ganancia en tiempo de consulta se vuelve mucho más eficiente.

Índices Multicolumna (Compuestos)

A menudo, una consulta en las tablas de una base de datos implica varias columnas de datos para mostrar la información. En estos casos, los índices de una sola columna no ofrecen un buen rendimiento. Debido a este impase, es necesario que tengamos múltiples índices de columnas a nuestro favor, lo cual es soportado por PostgreSQL, estando representado por la siguiente sintaxis:

```
CREATE INDEX index_name ON table_name (column1, column2);
```

PostgreSQL permite que estos índices se construyan sobre hasta 32 columnas, aunque en la práctica rara vez se utilizan tantas. Lo importante es el orden de las columnas en el índice, ya que el motor del sistema de base de datos las evaluará en ese orden jerárquico. Esto quiere decir que el índice funcionará mejor si las columnas del WHERE coinciden con las primeras columnas definidas en el índice.

Ejemplo:

Para que podamos ver mejor el uso del índice de múltiples columnas para optimizar las consultas, realizaremos la consulta donde obtendremos el número total de registros cuyo `codigo_producto` será menor a 400 y el precio del producto será menor a 300. Pero antes de realizar esta operación, hagamos una consulta simple sin usar el índice de la tabla para ver el tiempo necesario para realizar la consulta, como podemos ver a continuación:

```
SELECT COUNT(*) FROM produtos WHERE codigo_producto < 400 AND precio < 300;
```



Como podemos ver en la Ilustración 4, obtuvimos un tiempo de respuesta de 71 ms.

Query Query History Scratch Pad x

```
1 SELECT COUNT(*) FROM productos WHERE codigo_producto < 400 AND precio < 300
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	count bigint
1	1676

Total rows: 1 Query complete 00:00:00.071

✓ Successfully run. Total query runtime: 71 msec. 1 rows affected. ✕

CRLF Ln 1, Col 67

Ilustración 4 Consulta con múltiples columnas sin índice.

Veamos ahora la creación del índice compuesto por las columnas `codigo_producto` y `precio` para realizar la consulta. El índice se definirá de la siguiente manera:

```
CREATE INDEX productos_multicolumns_index ON productos (codigo_producto, precio);
```

Query Query History Scratch Pad x

```
1 CREATE INDEX productos_multicolumns_index ON productos (codigo_producto, precio);
```

Data Output Messages Notifications

CREATE INDEX

Query returned successfully in 57 msec.

✓ Query returned successfully in 57 msec. ✕

Ilustración 5 Crear el índice multicolumna.

Y ahora, volvamos a ejecutar la consulta, como podemos ver debajo de la siguiente declaración:

```
SELECT COUNT(*) FROM productos WHERE codigo_producto < 400 AND precio < 300;
```

Nuevamente obtuvimos un resultado más rápido, donde se registró un tiempo de ejecución de 49 ms usando el índice, como podemos ver en la Ilustración 6.

Query Query History Scratch Pad x

```
1 SELECT COUNT(*) FROM productos WHERE codigo_producto < 400 AND precio < 300;
```

Data Output Messages Notifications

Showing rows: 1 to 1 Page No: 1 of 1

	count bigint
1	1676

Total rows: 1 Query complete 00:00:00.049

✓ Successfully run. Total query runtime: 49 msec. 1 rows affected. ✕

CRLF Ln 1, Col 67

Ilustración 6 Resultado de la consulta con múltiples columnas con índice.



Para continuar con nuestro propósito de estudiar el uso de índices, crearemos una nueva base de datos, que debería llamarse dvdrental. Esta es una base de datos de ejemplo, disponible para pruebas con PostgreSQL, cuyo enlace de descarga es el siguiente: <https://neon.com/postgresql/postgresql-getting-started/postgresql-sample-database>

1. Creando la base de datos

Para comenzar, es necesario crear una base de datos vacía llamada dvdrental. Para ello, puedes ejecutar el siguiente comando SQL en el Query Tool de pgAdmin:

```
CREATE DATABASE dvdrental;
```

Una vez creada la base de datos, procederemos a importar los datos desde un archivo de respaldo.

2. Preparando el archivo de restauración

Al descargar el respaldo, recibirás un archivo comprimido en formato .rar, debes descomprimir ese archivo (puedes usar herramientas como WinRAR o 7-Zip). Dentro del .rar encontrarás un archivo con extensión .tar, que es el archivo de respaldo real que usaremos para restaurar la base de datos.

3. Restaurando la base de datos

- Abre pgAdmin4.
- Haz clic derecho sobre la base de datos dvdrental que acabas de crear.
- Selecciona la opción "Restore".

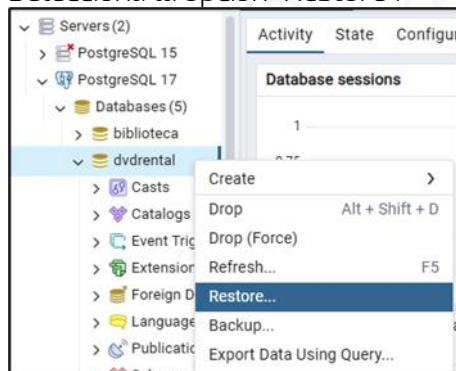


Ilustración 7 Restaurar una base de datos en PgAdmin4.

- Se desplegará la siguiente ventana:

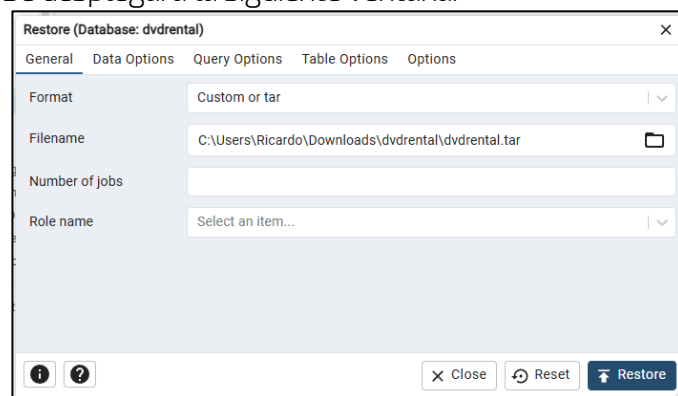


Ilustración 8 Restaurar una base de datos en PgAdmin4.



- Verifica que el formato de restauración esté en modo "Custom or tar".
- Ahora para seleccionar el archivo .tar, se nos abrirá la ventana en la cual veras un cuadro como el siguiente:

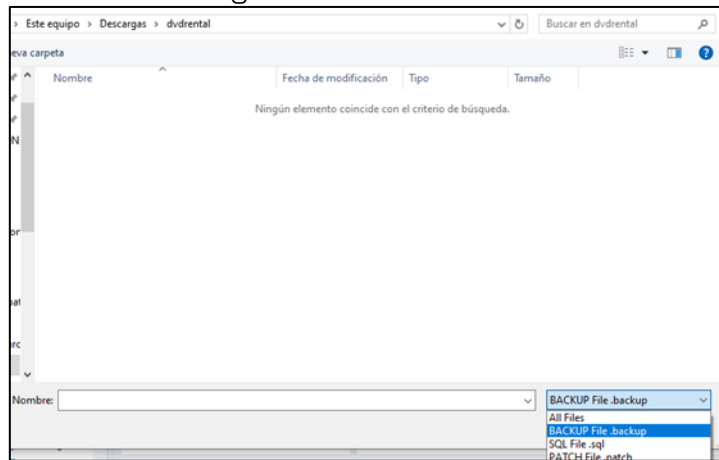


Ilustración 9 Ventana para seleccionar el archivo .tar..

- En este cuadro, asegúrate de seleccionar la opción "All Files" para que puedas ver y elegir el archivo .tar.
- Si dejas seleccionada la opción por defecto (por ejemplo, "BACKUP File .backup"), no podrás visualizar el archivo .tar y parecerá que no existe.

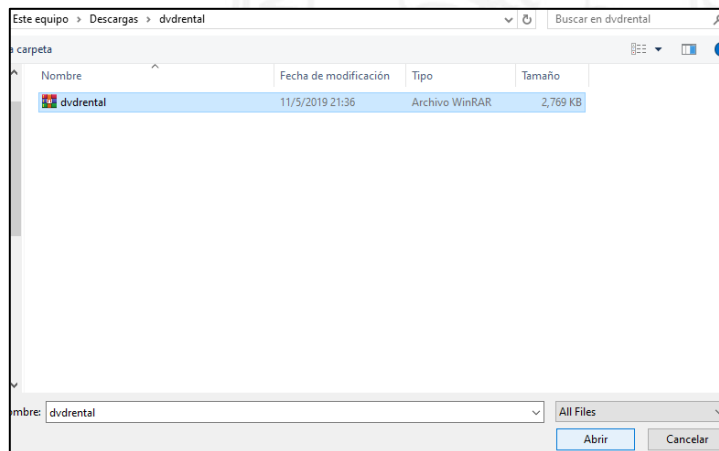


Ilustración 9 Archivo .tar seleccionado para restore en PgAdmin4.

- Finalmente, haz clic en el botón "Restore" para iniciar el proceso de restauración, una vez completada la restauración se te mostraran los siguientes mensajes:

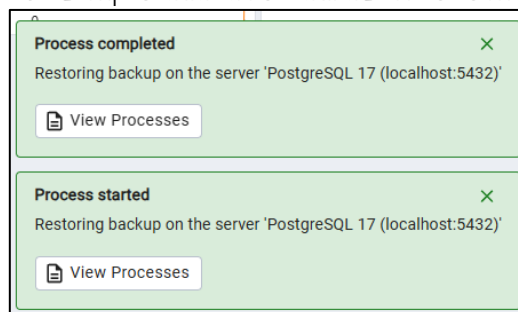


Ilustración 10 Confirmación de restore realizado correctamente en PgAdmin4.



Índice único

En cuanto a los índices exclusivos o únicos, estos se pueden crear para cualquier columna perteneciente a la tabla ya que no sólo crean índices, sino que refuerzan la exclusividad de la columna. El uso de este tipo de índice es ventajoso por razones de integridad de los datos y también por razones de rendimiento, lo que demuestra que las búsquedas que contienen índices exclusivos son mucho más rápidas.

Para que podamos crear un índice único tenemos varias formas, una de ellas es usando el comando `CREATE INDEX UNIQUE`, creando una restricción exclusiva en la tabla, o también se puede crear como clave primaria.

A continuación, mostramos cómo podemos crear un ejemplo de índice único usando `UNIQUE` para la tabla de clientes, presente en la base de datos de `dvdrental`:

```
CREATE UNIQUE INDEX customer_unique_index ON customer (customer_id);
```

La forma que presentamos anteriormente es una forma **explícita** de crear un índice exclusivo, donde usamos la palabra clave `UNIQUE`, pero también se puede crear implícitamente simplemente declarando una clave principal para la tabla.

A continuación, se muestra un ejemplo de una creación **implícita** de un índice único, que crea una clave principal para la tabla. Para demostrar la creación del índice implícitamente, podemos seguir las siguientes instrucciones:

```
ALTER TABLE customer ADD CONSTRAINT primary_key UNIQUE (customer_id);
```

Según la declaración anterior, modificamos la tabla y le agregamos una restricción única en la columna `customer_id` de la tabla de clientes, y esta declaración también crea implícitamente un índice único. El comando `ALTER` agrega una restricción única a la columna `customer_id`, que puede usarse como clave principal.

Índice de expresión

Los índices de expresión son útiles para consultas que corresponden a alguna función o modificación de nuestros datos en la tabla de la base de datos. De esta forma, Postgres nos permite indexar los resultados de esta función para que las búsquedas sean más eficientes, lo que, por ejemplo, puede ocurrir en momentos en los que pretendemos buscar por el nombre del cliente, donde la forma estándar de realizar esta operación es como:

```
SELECT * FROM customer WHERE LOWER(first_name) LIKE 'kimberly';
```

Según la consulta anterior, escaneamos cada fila de la tabla, convertimos el nombre a minúsculas y luego lo comparamos con "kimberly".

Entonces, veamos cómo crear una expresión índice para crear un índice en la columna `first_name`:

```
CREATE INDEX customer_expression_index ON customer (LOWER(first_name));
```

Según el índice creado, se realizará una búsqueda en la tabla de clientes según el nombre registrado. Como el nombre puede haber sido almacenado con letras mayúsculas y minúsculas, forzamos que la búsqueda de nombres en el índice sea siempre minúscula, lo que requiere el uso de índices de expresión. Un índice de expresión se utiliza sólo cuando se utiliza la expresión exacta en una consulta.



Índices parciales

Cuando tratamos con índices parciales, este tipo de índice cubre solo un subconjunto de datos que pertenecen a la tabla, donde se declara una cláusula WHERE. Con esto tenemos un aumento en la eficiencia del índice, ya que reducimos el tamaño del conjunto de datos a buscar.

Como ejemplo sencillo, podemos ver la creación del índice:

```
CREATE INDEX idx_fk_category_id ON film_category USING btree (film_id) WHERE category_id = 13;
```

Como se puede ver en las instrucciones anteriores, hemos creado un índice que trae un grupo reducido de información en función del código de categoría ingresado, debido al uso de la condición WHERE.

Sección 3: Reorganización de índices

El mantenimiento y reorganización de los índices es una tarea crítica para conservar el rendimiento y la consistencia en las bases de datos, especialmente en PostgreSQL, donde la arquitectura MVCC (Control de Concurrencia Multiversión) puede provocar que los índices acumulen información innecesaria (índices "hinchados").

Una de las preguntas que nosotros, los administradores de bases de datos, debemos tener en cuenta es cómo debemos lidiar con el llamado índice hinchado en las tablas de bases de datos PostgreSQL.

Llegados a este punto tenemos a nuestra disposición que la arquitectura MVCC (Multi-Version Concurrency Control) de PostgreSQL proporciona seguridad adicional a la hora de monitorear y mantener bases de datos, especialmente en sistemas con una gran cantidad de registros.

¿Pero qué es MVCC?

Cuando hablamos de MVCC, nos referimos a un método PostgreSQL utilizado para tratar la coherencia de los datos cuando varios procesos acceden a la misma tabla. En lugar de bloquear filas, PostgreSQL crea múltiples versiones de los datos, lo que permite operaciones concurrentes sin conflictos.

Sin embargo, este mecanismo genera acumulación de versiones antiguas que pueden inflar tanto las tablas como los índices, lo que se conoce como "bloat" (hinchamiento). Este fenómeno hace que los índices crezcan más allá de lo necesario, afectando el rendimiento y el uso de espacio en disco.

Cuando los índices crecen innecesariamente debido a múltiples actualizaciones, inserciones o eliminaciones, pueden volverse menos eficientes. Para resolver esto, se recomienda reorganizar o reconstruir los índices periódicamente.

ALTER INDEX

El comando ALTER INDEX es una sentencia de lenguaje SQL que se utiliza para modificar ciertas propiedades o configuraciones de un índice que ya ha sido creado. A diferencia de CREATE INDEX, que se usa para crear un índice desde cero, ALTER INDEX actúa sobre un índice existente, permitiendo realizar cambios no destructivos.



ALTER INDEX no modifica los datos de la tabla que está indexada, solo cambia aspectos relacionados con el propio índice.

Consideraciones sobre bloqueos

Cuando se ejecuta una operación con ALTER INDEX, el sistema debe garantizar la integridad del índice. Por eso, se aplican diferentes tipos de bloqueos mientras se ejecutan las modificaciones.

El tipo de bloqueo necesario varía según la operación:

- Algunas operaciones requieren bloqueos ligeros como SHARE UPDATE EXCLUSIVE, que permiten seguir leyendo datos mientras se realiza el cambio.
- Otras pueden requerir bloqueos más restrictivos como ACCESS EXCLUSIVE, que impiden cualquier acceso al índice hasta que la operación termine.

Si se usan múltiples subcomandos dentro de una misma instrucción ALTER INDEX, el sistema aplicará el bloqueo más restrictivo requerido por alguno de ellos.

Subcomandos de ALTER INDEX

PostgreSQL proporciona múltiples formas de utilizar ALTER INDEX, cada una con un propósito específico:

1. Renombrar un índice

El subcomando RENAME TO permite cambiar el nombre de un índice existente, esta operación es útil cuando se desea mejorar la claridad del esquema de la base de datos, alinear la nomenclatura con nuevas convenciones o simplemente corregir un nombre mal definido durante la creación inicial del índice. Al cambiar el nombre el índice sigue funcionando exactamente igual no se modifica su estructura, ni su contenido, ni su vinculación con la tabla correspondiente.

En PostgreSQL, si el índice está asociado a una restricción de integridad (como una clave primaria PRIMARY KEY, una restricción de unicidad UNIQUE, o una restricción de exclusión EXCLUDE), el sistema actualizará automáticamente el nombre de la restricción para reflejar el nuevo nombre del índice. Esto garantiza coherencia y evita inconsistencias en el catálogo del sistema.

Por ejemplo:

si se tiene un índice llamado idx_usuarios_email, y se desea renombrarlo por idx_email_usuarios para seguir una nueva convención de nombres, se puede utilizar el siguiente comando:

```
ALTER INDEX idx_usuarios_email RENAME TO idx_email_usuarios;
```

Esta operación requiere un tipo de bloqueo SHARE UPDATE EXCLUSIVE, que impide que otras transacciones modifiquen el índice mientras se renombra, pero no bloquea completamente el acceso a la tabla.



2. Mover un índice a otro tablespace

El subcomando SET TABLESPACE se emplea para mover un índice de un tablespace a otro, en PostgreSQL un tablespace es un área del sistema de archivos donde se almacenan físicamente los objetos de la base de datos. Esta característica permite distribuir el almacenamiento de la base de datos en diferentes discos o particiones, lo cual puede ser útil para mejorar el rendimiento o gestionar mejor el uso del espacio.

Mover un índice a un tablespace diferente puede ser una decisión estratégica. Por ejemplo, si se tiene un disco más rápido (como una unidad SSD), puede ser beneficioso almacenar allí los índices más utilizados, mientras que los índices menos críticos se pueden mantener en discos más lentos o económicos. También se puede realizar este movimiento por motivos de mantenimiento o reorganización física del sistema.

Ejemplo:

```
ALTER INDEX idx_pedidos_fecha SET TABLESPACE rapido;
```

Este comando moverá el índice idx_pedidos_fecha al tablespace denominado rapido, que debe haber sido previamente creado en el sistema. Es importante tener en cuenta que esta operación puede consumir recursos y tiempo, especialmente si el índice es grande, por lo que conviene realizarla durante períodos de baja actividad.

3. Modificar parámetros de almacenamiento del índice

Algunos tipos de índices permiten configurar ciertos parámetros de almacenamiento para ajustar su comportamiento. El más común de estos parámetros es fillfactor. El fillfactor determina el porcentaje de espacio que se llenará en cada página del índice al momento de su creación o reconstrucción, dejando el resto del espacio libre para futuras inserciones.

Un fillfactor menor implica que se reservará más espacio libre, lo cual puede reducir la fragmentación y mejorar el rendimiento en índices que se modifican con frecuencia. En cambio, un valor más alto maximiza el uso del espacio, pero puede afectar negativamente el rendimiento si el índice se actualiza o inserta con frecuencia.

Por ejemplo, el siguiente comando establece un fillfactor del 75% para un índice:

```
ALTER INDEX idx_ventas SET (fillfactor = 75);
```

Es importante aclarar que modificar este parámetro no cambia el índice de manera inmediata, el nuevo valor se aplicará solamente cuando el índice sea reconstruido, lo cual se puede hacer con el comando REINDEX. Sin esta acción adicional, el índice continuará utilizando la configuración anterior.

4. Reconstruir un índice

Con el paso del tiempo un índice puede acumular fragmentación interna debido a múltiples inserciones, eliminaciones y actualizaciones, lo que puede degradar su rendimiento. También es necesario reconstruir un índice después de modificar ciertos parámetros de configuración, como el fillfactor. Para estos casos, PostgreSQL ofrece el comando REINDEX.



El comando REINDEX permite recrear completamente un índice, eliminando la versión anterior y generando una nueva, optimizada y limpia. Esta operación puede mejorar significativamente el rendimiento de consultas si el índice estaba deteriorado o desorganizado.

Por ejemplo:

```
REINDEX INDEX idx_ventas;
```

Este comando eliminará y recreará el índice idx_ventas, utilizando la configuración actual, durante esta operación, se requiere un bloqueo exclusivo del índice, por lo que debe planificarse cuidadosamente en ambientes con alta concurrencia.

Además REINDEX también se puede aplicar a una tabla completa, a una base de datos o incluso a todo el sistema, si se desea reconstruir múltiples índices en una sola operación.

5. Modificar el nivel de estadísticas en índices de expresión

PostgreSQL permite crear índices sobre expresiones además de columnas simples, esto es útil para optimizar consultas que incluyen cálculos, transformaciones o funciones sobre columnas. Por ejemplo, se puede indexar la expresión (z + t) si se utiliza frecuentemente en consultas:

```
CREATE INDEX idx_coordenadas ON datos (x, y, (z + t));
```

Para mejorar la capacidad del optimizador de consultas de PostgreSQL, es posible ajustar la cantidad de estadísticas recolectadas sobre estas columnas expresivas. Esto se realiza con el subcomando ALTER COLUMN ... SET STATISTICS, el cual permite establecer un valor numérico que representa el nivel de detalle en las estadísticas recolectadas.

Por ejemplo:

```
ALTER INDEX idx_coordenadas ALTER COLUMN 3 SET STATISTICS 1000;
```

Este comando indica que se debe recolectar un nivel alto de estadísticas sobre la tercera columna del índice (la expresión (z + t)), lo que puede ayudar al optimizador a elegir el mejor plan de ejecución en consultas complejas.

Es importante destacar que este tipo de configuración tiene sentido en escenarios donde la distribución de los datos es irregular o donde se requiere una planificación precisa para mantener el rendimiento.

Los subcomandos de ALTER INDEX permiten a los administradores y desarrolladores de bases de datos gestionar de forma flexible y controlada los índices existentes, sin necesidad de eliminarlos o recrearlos desde cero. Cada subcomando cumple una función específica, y su uso adecuado puede tener un impacto significativo en el rendimiento general del sistema.



Actividad Interactiva #2

Ejercicio de asociación.

A continuación, verás una lista de términos y una lista de definiciones. Asocia Cada Término con su Definición: Une cada término con la definición correcta.

Términos:

1. B-tree
2. Hash
3. GiST
4. GIN



Definiciones:

- A. Índice especializado en comparaciones por igualdad como =, no admite ordenamientos ni búsquedas por rango.
- B. Índice por defecto en PostgreSQL, es versátil y funciona bien con igualdad, rangos y ordenamientos.
- C. Utilizado para búsquedas complejas, especialmente en datos espaciales o de geometría. Admite operadores personalizados.
- D. Indexa elementos individuales dentro de estructuras complejas como arrays, JSONB o texto completo.

Instrucciones: Asocia cada término (1-4) con la definición correspondiente (A - D). Y escríbela a continuación.
Ej: "1 - B"

Sección 4: Reconstrucción de índices

A lo largo del ciclo de vida de una base de datos, los índices pueden degradarse por múltiples razones: inserciones masivas, actualizaciones frecuentes, eliminaciones, cambios en los parámetros de almacenamiento, entre otros. Esta degradación puede afectar negativamente el rendimiento de las consultas que dependen de dichos índices. Por esta razón, es fundamental contar con mecanismos que permitan reconstruir o regenerar los índices existentes de forma eficiente y segura.

PostgreSQL ofrece el comando REINDEX para estos fines, este comando permite reconstruir un índice existente sin necesidad de eliminarlo explícitamente y crearlo de nuevo. La reconstrucción del índice reemplaza la versión anterior por una nueva, más compacta y optimizada, utilizando la estructura actual de datos y las configuraciones definidas (como el fillfactor).

Comando REINDEX

REINDEX crea una nueva copia del índice, ordenada y libre de fragmentación, utilizando la misma definición que tenía el índice original. Durante este proceso, el sistema reemplaza internamente el índice viejo por el nuevo, lo cual puede mejorar significativamente la eficiencia en consultas y operaciones de búsqueda. Este comando es útil en los siguientes casos:

- Cuando un índice se ha vuelto ineficiente por fragmentación.
- Cuando se desea aplicar una nueva configuración del índice (por ejemplo, un nuevo fillfactor).
- Después de una recuperación del sistema o corrupción detectada en el índice.
- Como parte de tareas regulares de mantenimiento en bases de datos activas.



La sintaxis básica es:

```
REINDEX [ ( VERBOSE ) ] { INDEX | TABLE | SCHEMA | DATABASE | SYSTEM } name;
```

Componentes de la sintaxis:

- **VERBOSE (opcional):** Si se incluye esta palabra clave, el sistema mostrará un informe detallado del progreso de la operación, lo cual es útil para tareas de diagnóstico o auditoría.
- **INDEX:** Reconstruye un único índice.
- **TABLE:** Reconstruye todos los índices asociados a una tabla específica.
- **SCHEMA:** Reconstruye todos los índices de todas las tablas pertenecientes a un esquema.
- **DATABASE:** Reconstruye todos los índices definidos en una base de datos completa.
- **SYSTEM:** Reconstruye los índices del catálogo del sistema, es decir, aquellos que pertenecen a las tablas internas de PostgreSQL.

Ejemplos prácticos:

1. **Reconstruir un índice específico:**
REINDEX INDEX idx_ventas_fecha;
Este comando reemplaza el índice idx_ventas_fecha por una nueva versión más optimizada.
2. **Reconstruir todos los índices de una tabla:**
REINDEX TABLE ventas;
Se regeneran todos los índices definidos sobre la tabla ventas.
3. **Reconstruir todos los índices de un esquema:**
REINDEX SCHEMA ventas_publicas;
Se reconstruyen todos los índices que pertenecen a las tablas del esquema ventas_publicas.
4. **Reconstruir todos los índices de una base de datos:**
REINDEX DATABASE mi_base_datos;
Este comando regenera todos los índices de todas las tablas contenidas en la base de datos mi_base_datos.
5. **Reconstruir los índices del catálogo del sistema (sólo para superusuarios):**
REINDEX SYSTEM mi_base_datos;
Se regeneran los índices utilizados internamente por el sistema de PostgreSQL (catálogo del sistema), que contienen metadatos esenciales para el funcionamiento del motor de base de datos.

Diferencia entre REINDEX y DROP + CREATE INDEX

Una pregunta frecuente entre administradores de bases de datos es si no es preferible simplemente eliminar un índice y volver a crearlo. Ambos enfoques, REINDEX y la combinación DROP INDEX + CREATE INDEX, pueden generar el mismo resultado en cuanto a reconstrucción, pero existen diferencias importantes en su comportamiento, especialmente en lo que respecta al manejo de bloqueos y concurrencia.



REINDEX:

- Bloquea las escrituras sobre la tabla relacionada mientras se reconstruye el índice.
- No bloquea las lecturas de la tabla, lo que permite que las consultas sigan ejecutándose, aunque sin usar el índice que se está regenerando.
- Bloquea exclusivamente el índice que está siendo procesado, no la tabla completa (a menos que se use REINDEX TABLE).
- Permite una reconstrucción más segura y controlada, especialmente en entornos de producción donde es necesario mantener la disponibilidad.

DROP INDEX + CREATE INDEX:

- DROP INDEX bloquea completamente la tabla: impide tanto lecturas como escrituras, ya que adquiere un bloqueo ACCESS EXCLUSIVE, el más restrictivo.
- CREATE INDEX permite lecturas, pero no escrituras, durante el proceso de creación. Sin embargo, el proceso puede ser costoso si la tabla es grande, ya que requiere leer todos los datos sin poder escribir en ellos.
- Existe un periodo en que el índice no existe, lo cual puede afectar negativamente al rendimiento de las consultas que dependen de él.
- Es más riesgoso si la base de datos se encuentra en uso activo, ya que implica la desaparición temporal del índice.

